SECURITY REMEDIATION REPORT

Business Logic Protection — Server-Side Migration

PreciseDM Application

Date: March 26, 2026

Classification: CONFIDENTIAL

1. Executive Summary

A security audit identified that all four proprietary insulin dosing algorithms (DiaForm, Steroid, Maintenance, and Gestation) were executing entirely on the client side. This meant the complete calculation logic—including formulas, thresholds, dose ranges, and category codes—was bundled into the JavaScript delivered to every user's browser and mobile app.

The audit confirmed that a technical user could extract the .js bundle from both the website (via browser DevTools) and the iOS/Android app (from the .ipa/.apk), fully reverse-engineer the algorithms, and replicate them independently—bypassing the subscription and potentially reselling the logic.

All four calculators have now been migrated to execute server-side via a secure backend function. The client application no longer contains any calculation logic.

2. Vulnerability Assessment

2.1 Findings (Before Remediation)

Calculator	Exposure Vector	Risk Level	Status
DiaForm	JS bundle (web + app)	CRITICAL	REMEDIATED
Steroid	JS bundle (web + app)	CRITICAL	REMEDIATED
Maintenance	JS bundle (web + app)	CRITICAL	REMEDIATED
Gestation	JS bundle (web + app)	CRITICAL	REMEDIATED

2.2 Attack Vectors Identified

1. Web Browser: DevTools → Sources tab → bundled .js files contained all formulas, constants (0.15, 0.18, 142, 0.9938, 1800/TDD, etc.), and branching logic in readable form.

2. iOS App (.ipa): Standard extraction tools could unpack the Capacitor web assets and expose identical JavaScript.

3. Android App (.apk): APK decompilation exposed the same web bundle with all calculation logic.

4. Subscription Bypass: Since calculations ran locally, a user could call the functions directly without an active subscription.

3. Remediation Actions

3.1 Architecture Change

Before (Vulnerable)	After (Secured)
Client collects inputs → Client executes calculation → Client displays results	Client collects inputs → Server validates auth + subscription → Server executes calculation → Server saves submission → Client displays results only

3.2 Changes Made

Component	Action	Details
supabase/functions/calculate/index.ts	CREATED	New server-side edge function containing all 4 calculator algorithms
src/hooks/use-calculate.ts	CREATED	Client hook for authenticated API calls to the calculate function
src/pages/DiaFormPage.tsx	MODIFIED	Removed calculate() function and doseRanges constants. Replaced with server API call.
src/pages/SteroidPage.tsx	MODIFIED	Removed calculate() function and all dosing constants. Replaced with server API call.
src/pages/MaintenancePage.tsx	MODIFIED	Removed calculateMaintenance() and avgOfProvided(). Replaced with server API call.
src/pages/GestationPage.tsx	MODIFIED	Removed calculateGestation() and avgOfProvided(). Replaced with server API call.
src/hooks/use-save-submission.ts	DEPRECATED	No longer imported. Submissions are now saved server-side automatically.

3.3 Server-Side Security Controls

Authentication: Every calculation request requires a valid JWT token. Unauthenticated requests are rejected with HTTP 401.

Subscription Enforcement: The server verifies the user has an active subscription (or admin role) before processing. Unauthorized users receive HTTP 403.

Server-Side Submission Logging: All calculations are logged to the form_submissions table using a service role key, ensuring audit trail integrity.

Input Validation: The server validates form_type against a whitelist (diaform, steroid, maintenance, gestation) and rejects unknown types.

4. Verification

4.1 What the Client Now Contains

The client-side code now contains ONLY: form UI components, input validation (basic type checks for UX), API call to the server, and result display templates. **Zero calculation logic, zero dosing constants, zero formulas.**

4.2 How to Verify

1. Open the deployed website in a browser, go to DevTools → Sources, and search for any dosing constants (e.g., 0.15, 0.18, 1800, 0.9938). None should be found in the application bundle.

2. Extract the .ipa or .apk and inspect the web assets directory. The JavaScript bundle should contain no calculation functions.

3. Attempt to call the /functions/v1/calculate endpoint without a valid JWT. The server should return 401 Unauthorized.

4. Attempt to call with a valid JWT but no active subscription. The server should return 403 Forbidden.

5. Impact Assessment

Aspect	Before	After
IP Protection	**EXPOSED**	**PROTECTED**
Subscription Enforcement	**CLIENT-SIDE ONLY**	**SERVER-SIDE**
Audit Trail	Client-initiated (bypassable)	**SERVER-ENFORCED**
Internet Required	No (offline capable)	Yes (required for calculations)
Latency	Instant (local)	~100-300ms (network roundtrip)

Note: *The app already required internet for authentication and subscription validation. The calculation dependency on internet connectivity does not change the existing user experience.*

6. Recommendations

1. Rebuild & Redeploy: Run `npx cap sync ios` and `npx cap sync android`, then archive and distribute new builds to the App Store and Play Store.

2. End-to-End Testing: Test all four calculators with the same 20 validation cases used in the original audit to confirm output parity.

3. Rate Limiting: Consider adding rate limiting to the calculate endpoint to prevent abuse or automated extraction attempts.

4. Legal Protection: While the algorithms are now technically protected, consider pursuing patent protection for the dosing methodologies as an additional layer of IP defense.

— End of Report —